Homework 5 Solutions

Mohammad Abu Shariha

October 13, 2025

Problem 1. Strongly Connected Components (20 pts)

[cite$_s$*tart*]*WeuseKosarajusalgorithmonthegraphsfromtheassignmentsheet*[*cite* : 138].[*cite*$_s$*tart*]*Forany*
139].

(a) In what order are the SCCs found?

Graph (i)

1. DFS on G: A systematic traversal (starting from A, exploring neighbors alphabetically) yields the following finishing order, from last to first:

Finish order (high to low): $\texttt{E}, \texttt{B}, \texttt{A}, \texttt{C}, \texttt{D}, \texttt{F}, \texttt{H}, \texttt{J}, \texttt{I}, \texttt{G}$

2. DFS on G^R: We process vertices in decreasing order of finishing times (E, B, A, ...).

- Start DFS from `E`: finds SCC $\{\texttt{E}\}$
- Start DFS from `B`: finds SCC $\{\texttt{B}\}$
- Start DFS from `A`: finds SCC $\{\texttt{A}\}$
- Start DFS from `C`: finds SCC $\{\texttt{C}\}$
- Start DFS from `D`: finds SCC $\{\texttt{D}\}$
- Start DFS from `F`: finds SCC $\{\texttt{F}\}$
- Start DFS from `H`: finds SCC $\{\texttt{H}\}$
- Start DFS from `J`: finds SCC $\{\texttt{J}\}$
- Start DFS from `I`: explores `G`, which explores `I` (already in stack). Finds SCC $\{\texttt{G}, \texttt{I}\}$

Final SCCs found (in order):

$$\{\texttt{E}\}, \{\texttt{B}\}, \{\texttt{A}\}, \{\texttt{C}\}, \{\texttt{D}\}, \{\texttt{F}\}, \{\texttt{H}\}, \{\texttt{J}\}, \{\texttt{G}, \texttt{I}\}$$

Graph (ii)

1. DFS on G: A traversal starting from A yields the finish order:

Finish order (high to low): $\texttt{B}, \texttt{C}, \texttt{A}, \texttt{D}, \texttt{E}, \texttt{F}, \texttt{G}, \texttt{H}, \texttt{I}$

2. DFS on G^R: Processing vertices in this order reveals that there are no cycles between vertices (only a self-loop on H). Therefore, every vertex forms its own SCC. **Final SCCs found (in order):**

$$\{\texttt{B}\}, \{\texttt{C}\}, \{\texttt{A}\}, \{\texttt{D}\}, \{\texttt{E}\}, \{\texttt{F}\}, \{\texttt{G}\}, \{\texttt{H}\}, \{\texttt{I}\}$$

[cite$_s$*tart*]

(b) Which are source and sink SCCs? [cite: 163]

Graph (i):

- **Source SCCs** (no incoming edges in metagraph): {E}, {B}
- **Sink SCCs** (no outgoing edges in metagraph): {J}

Graph (ii):

- **Source SCCs**: {B}
- **Sink SCCs**: {I}

[cite$_s tart$]

(c) Metagraphs [cite: 164]

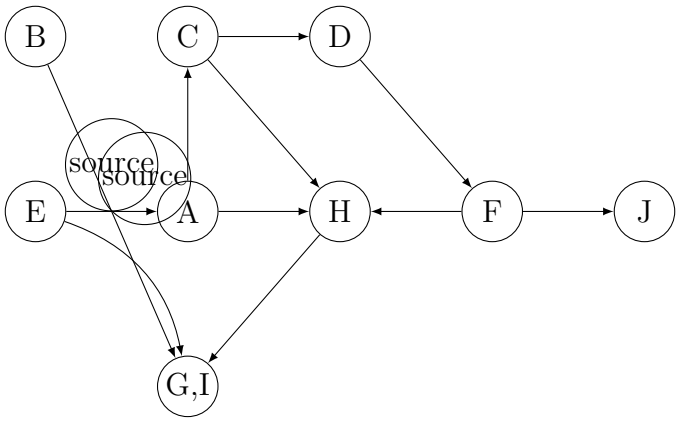


Figure 1: Corrected Metagraph for Graph (i)

[cite$_s tart$]

(d) Minimum edges to make graph strongly connected [cite: 167]

The formula is max(#source SCCs, #sink SCCs).

- **Graph (i):** $\max(2, 1) = \mathbf{2}$ edges needed.
- **Graph (ii):** $\max(1, 1) = \mathbf{1}$ edge needed.

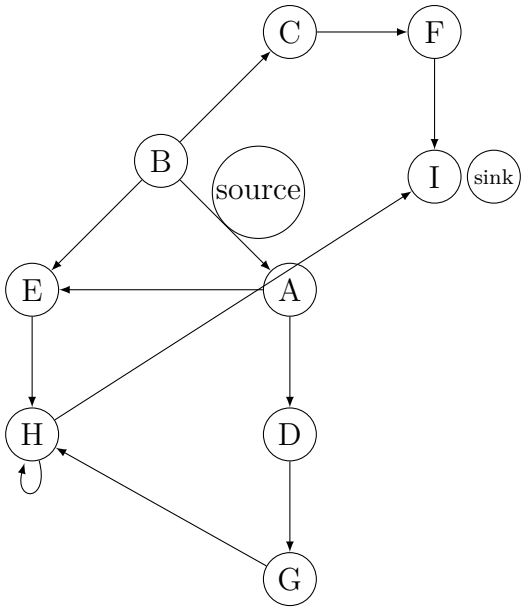


Figure 2: Corrected Metagraph for Graph (ii)

[cite$_s tart$]

Problem 2. Universal Reachability Algorithm (20 pts) [cite: 168]

An efficient algorithm to determine if a vertex s exists from which all other vertices are reachable is as follows:

Algorithm:

1. Run Kosaraju's algorithm or Tarjan's algorithm on the graph $G = (V, E)$ to find all of its Strongly Connected Components (SCCs).
2. Construct the component graph, G^{SCC}, where each node represents an SCC from step 1. An edge exists from SCC C_i to C_j in G^{SCC} if there is an edge in G from a vertex in C_i to a vertex in C_j.
3. Count the number of "source" SCCs in G^{SCC}. A source SCC is one with an in-degree of 0.
4. If there is **exactly one** source SCC, then such a vertex s exists (any vertex within that single source SCC will work). Otherwise, no such vertex exists.

Running Time: The running time is dominated by finding the SCCs.

- Step 1 (Kosaraju's algorithm) takes $O(|V| + |E|)$ time.

- Step 2 (Building G^{SCC}) takes $O(|V| + |E|)$ time by iterating through all vertices and edges once.
- Step 3 (Counting source SCCs) takes $O(|V| + |E|)$ time.

The total running time is $\mathbf{O}(|\mathbf{V}| + |\mathbf{E}|)$.

Correctness: The component graph G^{SCC} is a Directed Acyclic Graph (DAG). For a vertex s to reach all other vertices in G, it must belong to an SCC that can reach all other SCCs in G^{SCC}. This is only possible if s is in a source SCC. If there is more than one source SCC, vertices in one source component cannot reach vertices in another, so no single vertex can reach everyone. If there are zero source SCCs, it implies a cycle in the component graph, which is impossible. Therefore, a universal starting vertex s exists if and only if there is exactly one source SCC.

[cite$_start$]

Problem 3. Unique Shortest Path (20 pts) [cite: 170]

Idea: We can modify Dijkstra's algorithm to not only track the shortest distance to each vertex but also to count the number of distinct shortest paths. [cite$_start$]$Apathfromstouisuniqueifthiscountis$ 174].

Algorithm:

1. Initialize arrays: $d[v] = \infty$ for all $v \in V$, and $count[v] = 0$ for all $v \in V$.
2. Set source conditions: $d[s] = 0$ and $count[s] = 1$. [cite$_start$]$Initializeabooleanarray$`usp`[s] = true[$cite$: 175].
2. Use Dijkstra's algorithm, typically with a priority queue, starting from s.
3. When relaxing an edge (u, v) with weight $l(u, v)$:
    - **If a shorter path is found** ($d[u] + l(u, v) < d[v]$): Update the distance and reset the path count.
    $$d[v] = d[u] + l(u, v) \quad \text{and} \quad count[v] = count[u]$$
    - **If another path of the same length is found** ($d[u] + l(u, v) = d[v]$): Add the number of paths from u.
    $$count[v] = count[v] + count[u]$$
4. After Dijkstra's terminates, iterate through all vertices $v \in V$. Set `usp`$[v]$ = (`count`$[v]$ == 1).

Time Complexity: The modifications add only a constant number of operations within the relaxation step of Dijkstra's algorithm. [cite$_start$]$Therefore, thetimecomplexityremainsthesameasDi$ 171].

Correctness: The algorithm correctly counts paths because of the properties of shortest paths in graphs with non-negative edges. When we extract a node u from the priority queue, we have found its final shortest distance. Any path to a neighbor v through u will be built upon one of the shortest paths to u. By summing the counts when we find paths of equal length ($d[v] = d[u] + l(u, v)$), we correctly aggregate all distinct shortest paths to v. A final count of 1 for a vertex u guarantees that only one such path was ever found.

[cite$_start$]

Problem 4. Dijkstra's Algorithm with Negative Edges (20 pts) [cite: 176]

Answer: No, Dijkstra's algorithm, started at s, will **not fail** on such a graph. It will produce the correct shortest path distances.

Proof / Argument: The correctness of Dijkstra's algorithm relies on the assumption that once a vertex u is extracted from the priority queue, its shortest path distance $d[u]$ is finalized and will never be updated again. This holds true if all edge weights are non-negative, because any alternative path to u would have to go through another vertex v still in the queue, and since $d[v] \geq d[u]$, the path length could not be shorter.

This guarantee is threatened by negative edges. A negative edge could potentially create a shorter path to an already-visited vertex. Let's analyze the specific conditions given:

1. Only edges leaving the source s, (s, v), can be negative[cite: 176]. [cite$_start$]

1. All other edges in the graph have positive weights[cite: 177]. [cite$_start$]

1. The source s has no incoming edges[cite: 178].

Execution Trace:

- **Step 1 (Initialization):** Dijkstra's initializes $d[s] = 0$ and adds s to the priority queue.
- **Step 2 (Processing s):** s is extracted. For each neighbor v of s, the algorithm sets $d[v] = l(s, v)$. This value can be negative. All neighbors of s are added to the priority queue.
- **Step 3 (Main Loop):** The algorithm now proceeds to extract vertices from the priority queue. Let the next vertex extracted be u. For any path from u to another vertex w, the path will be of the form $s \to \cdots \to u \to \cdots \to w$.

The crucial observation is that any edge (x, y) on a path *after* the initial step from s must have $x \neq s$. According to the problem statement, all such edges are positive.

Since all remaining edges to be explored have positive weights, the core assumption of Dijkstra's algorithm holds for the remainder of its execution. A path to an already-finalized vertex cannot be improved, because doing so would require traversing a sequence of positive-weight edges, which can only increase the path length. Since s has no incoming edges, no path can ever loop back to s to take advantage of the negative edges again.

Therefore, Dijkstra's algorithm works correctly under these specific constraints.

[cite$_s tart$]

Problem 5. BFS on Directed Graphs (20 pts) [cite: 179]

[cite$_s tart$]

(a) Why no BFS forward edges? [cite: 186]

In a Breadth-First Search (BFS), the graph is explored level by level starting from a source s. Let $level[u]$ be the length of the shortest path from s to u. By the definition of BFS, for any edge $(u, v) \in E$, the level of v can be at most one greater than the level of u. That is:

$$level[v] \leq level[u] + 1$$

[cite$_s tart$]$ABFSforwardedgeisdefinedasanedgefromanodeutoanon-childdescendantvintheBFStree[cit$
183].$Forvtobeadescendantofu, itslevelmustbegreaterthanu'slevel.Foritobeanon\text{-}childdescendant, itsleve$
$level[u]$+1 This directly contradicts the fundamental property of BFS ($level[v] \leq level[u]$+1).
If such an edge (u, v) existed, BFS would have discovered v from u at level $level[u] + 1$, a contradiction. Therefore, **BFS forward edges are impossible**.

[cite$_s tart$]

(b) Classification Algorithm [cite: 187]

We can classify all edges by first running BFS to determine the level and parent of each node, and then iterating through all edges.

Algorithm:

1. Initialize `level`$[v] = \infty$ and `parent`$[v] =$ NIL for all $v \in V$.
2. Run BFS starting from a source s. During the traversal, when exploring from vertex u to an unvisited neighbor v, set:
   - `level`$[v] =$ `level`$[u] + 1$
   - `parent`$[v] = u$
3. Iterate through every edge $(u, v) \in E$ in the graph:
   - If `parent`$[v] == u$, classify (u, v) as a **Tree edge**[cite: 182]. [cite$_s tart$]
   - Else if $level[v] < level[u]$, classify (u, v) as a **Back edge**[cite: 184]. [cite$_s tart$]
   - Else (which implies $level[v] == level[u]$ or $level[v] == level[u]+1$), classify (u, v) as a **Cross edge**[cite: 185].

Time Complexity: Step 2 (BFS) takes $O(|V| + |E|)$ time. Step 3 involves a single pass through all edges, which takes $O(|E|)$ time. The total complexity is $\mathbf{O}(|\mathbf{V}| + |\mathbf{E}|)$.